

Survey Paper — SynapseOS: Designing a Conversational Interface Layer for Personal Computing

A Survey of Conversational and LLM-Mediated Interfaces for
Personal Computing: Toward SynapseOS

Willard C. Soriano

Department of Information Technology

Mapua University – Makati

wcsoriano@mymail.mapua.edu.ph

Table of Contents

Survey Paper — SynapseOS: Designing a Conversational Interface Layer for Personal Computing.....	1
Abstract.....	3
I. Introduction.....	3
II. Terminology.....	4
III. General Research Framework.....	6
IV. Paper Surveys.....	7
Group A: Historical Foundation.....	7
Group B: Shell and CLI Layer.....	8
Group C: Single-Application and Accessibility Layer.....	9
Group D: Web Agents.....	10
Group E: Mobile Agents.....	11
Group F: Vision-Language Foundation.....	13
Group G: Desktop and Cross-Application Layer.....	13
Group H: Evaluation and Synthesis.....	15
V. Analysis of Papers.....	15
A. Comparison Matrix.....	15
B. Cross-Paper Patterns.....	17
C. Tradeoffs Identified.....	19
VI. Proposed Conceptual Framework: SynapseOS.....	20
A. Architecture Overview.....	20
B. Differentiators.....	21
C. Evaluation Plan.....	21
D. Open Questions.....	22
References.....	23

Abstract

Despite four decades of research into natural-language interfaces to personal computing — from the symbolic-AI consultants developed for the Unix shell in the 1980s to the large language model (LLM) agents that now operate graphical user interfaces, browsers, and mobile applications — no working full-operating-system conversational replacement has been built and empirically evaluated against the graphical desktop it would displace, and Linux desktop coverage remains conspicuously absent from the literature. This survey reviews twenty works published between 1988 and 2025, organizing the field along a four-layer pipeline of Input, Reasoning, Action, and Operating-System Integration. After defining the principal terminology and research framework, each surveyed work is analyzed along scope, methodology, results, and open questions, and cross-paper patterns are synthesized to identify the gap motivating the proposed system: SynapseOS, a Linux-based conversational interface layer designed to function as the primary user-facing interface to a personal computing environment, with an evaluation methodology that engages the conventional desktop workflow as its baseline rather than as its substrate.

I. Introduction

The dominant paradigm for personal computing has remained largely unchanged since the early 1980s. Users interact with desktop operating systems through icons, menus, and pointer-driven graphical interfaces inherited from the Xerox Star and Apple Lisa lineage. Yet for almost as long, a parallel research thread has explored an alternative: that users should be able to describe what they want in natural language and have the computer carry it out. The earliest serious system in this tradition, the Berkeley UNIX Consultant project, was already operational by the late 1980s. Decades later, the rise of LLMs and multimodal foundation models has reopened this thread with new technical capability. The field that has emerged sits at the intersection of human-computer interaction, operating systems, natural language processing, and computer vision.

The current status of the field is one of rapid and uneven progress. On the shell side, recent work has shown that LLMs can translate natural language into Bash with usable accuracy and can be embedded as a Unix primitive with appropriate guardrails. On the graphical side, "computer-using agents" —

sometimes called LLM-brained GUI agents — can now operate desktop and mobile applications by combining native APIs with vision-based grounding. On the web, language-guided agents complete tasks across realistic websites with measurable but still limited reliability. Yet the most rigorous evaluation environments, including the OSWorld benchmark, demonstrate that frontier models complete only roughly one in eight real computer tasks that humans complete at over seventy percent. The gap between demonstration and deployable replacement remains large.

The seminal works of the field span two distinct eras. In the symbolic-AI era, the Berkeley UNIX Consultant of Wilensky and colleagues established the goal of a natural-language interface to a real operating system and developed the conceptual machinery — plan recognition, user modeling, knowledge representation — that justified pursuing it. In the LLM era, the introduction of GPT-4 with vision and the subsequent demonstrations that such models could be grounded into web interfaces (notably the SeeAct framework of Zheng and colleagues) marked the transition from research curiosity to engineering possibility. The field's current activity is concentrated in venues including NeurIPS, ICML, CVPR, ACL, CHI, UIST, OSDI, and USENIX ATC, with industrial laboratories (Microsoft Research, Google DeepMind, Anthropic, Tsinghua/Zhipu AI, Alibaba) contributing alongside academic groups at OSU, CMU, Berkeley, MIT, and elsewhere. Funding has flowed from both classical defense and commercial-AI sources.

The current research challenges are concrete. Visual grounding, the problem of mapping a textual plan onto the correct on-screen element, is repeatedly identified as the field's principal bottleneck. Long-horizon planning across many steps remains brittle. Safety and recoverability — what happens when an agent takes a destructive action — are underdeveloped. Evaluation methods do not yet support comparison against the conventional graphical workflow that conversational interfaces propose to replace. And almost all empirical work to date is Windows-, Android-, or browser-centric; Linux desktop coverage is conspicuously thin, despite Linux being the substrate on which most server-side and many developer workflows depend. SynapseOS, the system proposed at the conclusion of this survey, is positioned against these gaps.

II. Terminology

The following terms are defined as they are used in the surveyed literature and in the rest of this paper.

A **conversational interface** is a software system that accepts user input in natural language (text, speech, or both) and responds in natural language or by performing actions on the user's behalf. A **large language model (LLM)** is a transformer-based neural network trained on large text corpora that generates text continuations from a prompt. A **multimodal large language model (MLLM)**, also called a **vision-language model (VLM)**, additionally accepts images as input and is capable of producing text grounded in visual content.

A **computer-using agent (CUA)** is a system that uses an LLM or MLLM to operate a computing environment by issuing actions and observing results. A **GUI agent** is a CUA whose action space consists of operations on a graphical user interface, typically clicks, key presses, scrolls, and text entry. **Grounding** is the problem of mapping an LLM's textual plan onto a concrete operation in the environment; in GUI agents, **visual grounding** is the specific subproblem of identifying the on-screen pixel region corresponding to a referenced element.

The **action space** of an agent is the set of operations it is permitted to issue. **Set-of-Mark (SoM) prompting** is a visual technique in which candidate UI elements are overlaid with numbered marks so that a VLM can refer to them by index rather than by coordinates. The **Model Context Protocol (MCP)** is a recent standard for exposing application APIs and data sources to LLMs. The **accessibility tree** (a11y tree) is the structured representation of UI elements maintained by an operating system or browser for assistive technologies; many GUI agents read from it as a structured alternative to screenshots.

An **agent OS** is a system framework that organizes one or more LLM-powered agents around the abstractions of a conventional operating system. **Tool use**, also called **function calling**, is the practice of allowing an LLM to invoke external functions whose names and signatures are described in its prompt. **Task decomposition** or **planning** is the partitioning of a high-level instruction into a sequence of intermediate subgoals. A **long-horizon task** is one whose successful completion requires many sequential actions. A **trajectory** is the ordered sequence of observations and actions an agent produces during a task. The **document object model (DOM)** is the structured representation of a web page that a browser exposes to scripts and tools.

III. General Research Framework

The surveyed literature is organized along a four-layer pipeline, illustrated in Figure 1 (described in prose). Each layer corresponds to a distinct engineering and research concern, and each surveyed work occupies one or more of the layers.

Layer 1: Input. This layer concerns how the user expresses intent to the system. Modalities include typed text, spoken voice via automatic speech recognition, multimodal input that combines text or voice with pointing or visual context, and structured inputs such as macros or templates. Research questions at this layer include how to design prompts that elicit useful intent from users, how to handle ambiguity and underspecification, and how to accommodate users with motor, speech, or cognitive accessibility needs.

Layer 2: Reasoning. This layer is occupied by the LLM or MLLM and addresses how natural-language intent is parsed into structured plans, how those plans are decomposed into subgoals, how the system retrieves relevant context, and how it grounds its plans into the affordances of the environment it observes. The reasoning layer encompasses both task planning and the visual or symbolic grounding that connects the plan to what the system can actually do.

Layer 3: Action. This layer concerns how a grounded plan is executed against the underlying system. The literature exhibits three principal approaches: command translation, in which natural language is converted into shell or scripting commands; API binding, in which the agent invokes native application or operating-system APIs (often via accessibility trees, automation frameworks, or MCP); and GUI simulation, in which the agent emits mouse and keyboard operations inferred from screenshots. Hybrid approaches combine API binding with GUI simulation as a fallback.

Layer 4: Operating-System Integration. This layer concerns the scope of computing the agent can affect: a single application, the shell, a cross-application desktop, a mobile device, a browser-mediated web environment, or an entire operating system. Research questions at this layer include how to expose system primitives to agents safely, how to share context across applications, how to handle privilege and isolation, and how to deploy and evaluate agents at the OS level rather than per-application.

The four-layer framework is not original to this survey — analogous decompositions appear in the survey of LLM-brained GUI agents by Zhang and colleagues — but it is formalized here as the organizing schema for the per-paper analysis that follows.

IV. Paper Surveys

The twenty surveyed works are presented in eight thematic groups. Each entry follows the structure prescribed for survey papers: scope and dataset, methodology, results, and areas for further research.

Group A: Historical Foundation

[1] The Berkeley UNIX Consultant Project (Wilensky et al., 1988). *Scope and dataset.* UC was an intelligent natural-language interface that allowed naive users to learn about the UNIX operating system through conversation in ordinary English. The project addressed both the natural-language-frontend problem and the consultation problem, distinguishing itself from systems intended only as command translators. The "data" of the system was a hand-engineered knowledge base of UNIX concepts and English vocabulary. *Methodology.* The system was built on the KODIAK knowledge representation, a relation-oriented framework intended to support both wide representational range and clear semantics. UC included a user model component called KNOME that built representations of the user's knowledge state, and a teaching component called UCTeacher that allowed users to extend the knowledge base in natural language. *Results.* UC demonstrated end-to-end natural-language consultation about UNIX, successfully answering "how do I" questions, definitional questions, and problem-solving questions. The system was used as a research vehicle for plan recognition, user modeling, and knowledge representation throughout the late 1980s and into the 1990s. *Areas for further research.* The authors identified knowledge acquisition as the principal bottleneck — extending UC to a new domain required substantial hand-engineering. This concern foreshadows the contemporary problem of LLM-based agents requiring per-application grounding data.

[2] ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023). *Scope and dataset.* ReAct, published as a Notable Top 5% paper at ICLR 2023, introduces a prompting framework that interleaves reasoning traces with action invocations in language models. Evaluation spans four benchmarks: HotPotQA (question answering), Fever (fact verification), ALFWorld (text-based games),

and WebShop (webpage navigation). *Methodology.* The framework augments standard action-generation prompts with explicit free-text reasoning steps. The language model is prompted to alternate between "Thought" steps that update its plan and "Action" steps that interact with an external environment, with observations from the environment fed back into the next reasoning step. The combination is implemented purely as a prompting strategy without fine-tuning. *Results.* ReAct outperforms vanilla action-generation baselines on HotPotQA and Fever and is competitive with chain-of-thought prompting; the strongest configuration combines ReAct with chain-of-thought to use both internal and externally-retrieved knowledge. On ALFWorld and WebShop, even one-shot ReAct prompting substantially outperforms imitation-learning and reinforcement-learning baselines that require thousands of training trajectories. *Areas for further research.* The authors identify integration with parametric fine-tuning and scaling to longer-horizon tasks as directions; subsequently, the ReAct pattern has been adopted as an architectural primitive across nearly all LLM-agent systems including the GUI agents surveyed in Group G of this paper.

Group B: Shell and CLI Layer

[3] LLM-Supported Natural Language to Bash Translation (Westenfelder et al., 2025). *Scope and dataset.* The work concerns translation of natural language into Bash shell commands. The authors construct a manually verified test set of 600 instruction-command pairs and a training set of 40,939 pairs. *Methodology.* The work fine-tunes multiple LLMs on the training set and introduces a functional-equivalence heuristic that reaches 95% confidence in determining whether two Bash commands produce the same effect, allowing automated evaluation that goes beyond exact-string matching. *Results.* The fine-tuned models substantially outperform zero-shot baselines, demonstrating that fine-tuning specifically on command-translation data is effective. *Areas for further research.* Bash translation remains limited by ambiguity in user instructions and by the lack of system context (the contents of the current directory, the user's history, environment variables) at translation time. Integrating context-aware retrieval is identified as a direction.

[4] NaSh: Guardrails for an LLM-Powered Natural Language Shell (Gyawali et al., 2025). *Scope and dataset.* NaSh is a design proposal for a Linux natural-language shell that interposes guardrails between the user's natural-language input and the LLM-generated shell command. The work is published under the cs.OS classification on arXiv, signaling its positioning as operating-systems

research rather than NLP. *Methodology.* The authors propose a layered architecture in which the LLM proposes a candidate command, a guardrail layer evaluates the command against safety, reversibility, and confirmation criteria, and the user receives a structured preview before execution. *Results.* The work is presented through worked examples that illustrate the guardrail patterns. *Areas for further research.* The authors note that LLM outputs may be unintended or unexplainable, requiring guardrails for user error recovery; how to design recovery and rollback for irreversible system operations is left as open work, along with empirical user evaluation.

Group C: Single-Application and Accessibility Layer

[5] A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants (van Dam, 2025). *Scope and dataset.* The work proposes an architecture for retrofitting LLM-based conversational interfaces onto existing single-application graphical user interfaces. *Methodology.* The architecture exposes the application's navigation graph through the Model Context Protocol and uses the Model-View-ViewModel pattern, with each ViewModel publishing the tools relevant to its view. Voice input is handled by real-time speech recognition layered on top. *Results.* The work is an architectural prototype demonstrated on representative applications without a large-scale user study. *Areas for further research.* The author observes that most production applications were never designed with speech in mind; retrofitting voice and conversational interfaces onto applications with implicit visual conventions is identified as ongoing work.

[6] VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots (Padmanabha et al., 2024). *Scope and dataset.* VoicePilot, presented at UIST 2024, is a framework for using LLMs as speech interfaces to physically assistive robots used by individuals with motor impairments. While the immediate target is robotic feeding rather than desktop computing, the work is included here because it is the most rigorous human-centered evaluation in the LLM-speech-interface literature and informs the design of any conversational system intended for users with motor disabilities. *Methodology.* The framework defines a Robot API exposed in the LLM's prompt and a structured dialogue management layer; the authors derive design guidelines from a study with users with motor impairments. *Results.* The study yields five concrete design guidelines for LLM-based speech interfaces in assistive contexts, including the importance of execution speed approaching that of

a caregiver and the necessity of preview-and-confirm workflows. *Areas for further research.* The authors flag generalization across different user populations and across different physical tasks as open.

[7] DexAssist: A Voice-Enabled Dual-LLM Framework for Accessible Web Navigation (Mehendale and Walishetti, 2024). *Scope and dataset.* DexAssist targets web navigation by users with fine-motor impairments, focusing on e-commerce sites such as Amazon. *Methodology.* The framework uses two LLMs in tandem: one to interpret natural-language voice commands and produce a candidate target description, and a second to map that description onto DOM elements with fallback strategies when direct DOM matches fail. *Results.* The dual-LLM approach reduces the misinterpretation failures of single-LLM voice navigation, particularly on dense e-commerce layouts. *Areas for further research.* Generalization beyond e-commerce, integration with screen readers, and evaluation across a broader range of motor-impairment profiles are identified as future work.

Group D: Web Agents

[8] Mind2Web: Towards a Generalist Agent for the Web (Deng et al., 2023). *Scope and dataset.* Mind2Web, presented at NeurIPS 2023 as a Datasets and Benchmarks Spotlight, is the first dataset for developing generalist web agents that follow language instructions across diverse real-world websites. The dataset comprises over 2,000 open-ended tasks collected from 137 websites spanning 31 domains, with crowdsourced action sequences. *Methodology.* The authors introduce MINDACT, a two-stage model that uses a small language model to filter candidate elements from large HTML documents before invoking a larger LLM to choose among them. *Results.* MINDACT demonstrates non-trivial performance on websites and domains the model has not seen during training, establishing the feasibility of generalist web agents. The authors note substantial room for improvement toward truly generalizable agents. *Areas for further research.* HTML representation, action-space design, and cross-domain transfer are identified as open. The work has spawned follow-up benchmarks including Online-Mind2Web and Multimodal-Mind2Web.

[9] WebArena: A Realistic Web Environment for Building Autonomous Agents (Zhou et al., 2024). *Scope and dataset.* WebArena, presented at ICLR 2024, provides a fully self-hosted, reproducible web environment with functional clones of e-commerce, social-forum, collaborative-software, and content-management websites, enriched with auxiliary tools and knowledge bases. The benchmark consists of 812 long-horizon tasks evaluated for functional correctness rather than trajectory

similarity. *Methodology.* The environment is implemented as Docker-deployed websites; agents interact via observable state and a defined action space; success is determined by post-task state evaluation. *Results.* GPT-4 baselines achieved low absolute completion rates at the time of publication, with completion rates substantially below human performance, establishing WebArena as a challenging benchmark. *Areas for further research.* Long-horizon planning, memory architectures, and skill abstraction across episodes are repeatedly identified as needed advances.

[10] GPT-4V(ision) is a Generalist Web Agent, if Grounded (Zheng et al., 2024). *Scope and dataset.* This work, presented at ICML 2024, introduces SeeAct, a framework for using GPT-4V as a generalist web agent. Evaluation is conducted on the Mind2Web benchmark and on a new live-website online evaluation harness. *Methodology.* SeeAct decouples visual understanding from action: the LMM observes a webpage screenshot, generates a textual plan, and a separate grounding step maps the plan onto HTML elements through one of three strategies (textual choices, element attributes, or image annotation). *Results.* With oracle grounding provided, GPT-4V completes 51.1% of tasks on live websites, substantially outperforming text-only GPT-4 (13.3%) and smaller fine-tuned models. Without oracle grounding, performance degrades, confirming grounding as the principal bottleneck. *Areas for further research.* The authors identify visual grounding for web elements as the principal open problem, motivating the subsequent SeeAct-V and UGround lines of work that train dedicated grounding models on synthetic GUI data.

Group E: Mobile Agents

[11] Android in the Wild: A Large-Scale Dataset for Android Device Control (Rawles et al., 2023). *Scope and dataset.* Presented at NeurIPS 2023 in the Datasets and Benchmarks track, AITW is a dataset of 715,000 episodes spanning 30,000 unique natural-language instructions, four versions of Android (10 through 13), and eight Pixel device types. *Methodology.* Each episode contains a goal instruction and a sequence of observation-action pairs, where observations are screenshots and actions are gestures (taps, drags) at arbitrary screen coordinates rather than UI-element-indexed actions. Hindsight relabeling is used to derive single-step training data from multi-step episodes. *Results.* The dataset establishes a substantial scale advantage over prior mobile-control datasets and exposes the challenge that actions must be inferred from visual appearance rather than from a structured UI model.

Areas for further research. Cross-app generalization, robustness to UI updates, and the action-space tradeoff between gestures and structured operations remain open.

[12] AppAgent: Multimodal Agents as Smartphone Users (Zhang et al., 2023). *Scope and dataset.* AppAgent is a multimodal agent framework for operating Android applications. Evaluation comprises 50 tasks across 10 applications including social-media, productivity, and entertainment apps. *Methodology.* The agent operates through a simplified action space mimicking human interactions (taps and swipes) and bypasses the need for system back-end access. The system labels manipulable UI regions with semi-transparent tags drawn from Android XML files. The agent learns to use new applications through autonomous exploration or by observing human demonstrations, generating a knowledge base of application-specific documentation. *Results.* Documents generated through self-exploration and demonstration matched or exceeded the performance of human-written documentation, demonstrating the viability of automated knowledge acquisition for mobile agents. *Areas for further research.* Generalization across application updates, handling of sequential dependencies between apps, and evaluation against more diverse user populations are identified as future work.

[13] Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception (Wang et al., 2024). *Scope and dataset.* Mobile-Agent introduces a vision-centric mobile device agent that does not require XML files or system metadata, in contrast to AppAgent. The Mobile-Eval benchmark is introduced to evaluate mobile device operations. *Methodology.* The agent uses visual-perception tools (an OCR module and an icon-detection module) to identify and locate visual and textual elements in app interfaces. Based on perceived visual context, the agent autonomously plans, decomposes, and navigates through application operations step by step. *Results.* The vision-centric approach enables operation across diverse mobile environments without per-system customization, at the cost of vision-grounding errors that require recovery mechanisms. *Areas for further research.* The work motivates Mobile-Agent-v2 (multi-agent collaboration) as a follow-up. Robustness to visual variation across Android skins and app updates remains open.

[14] Mapping Natural Language Instructions to Mobile UI Action Sequences (Li et al., 2020). *Scope and dataset.* This work, presented at ACL 2020, introduces the problem of grounding natural-language instructions to mobile user-interface actions and contributes three datasets: AndroidHowTo (annotated action-phrase spans in how-to instructions), Rico-SCA (synthesized grounded action descriptions for mobile UIs), and PixelHelp (a corpus pairing English instructions with action

sequences performed by people on a mobile UI emulator). The work predates the LLM era and is the conceptual ancestor of subsequent mobile-agent benchmarks including Android in the Wild. *Methodology.* The system uses a two-stage Transformer architecture: a phrase-tuple-extraction Transformer identifies action phrases in long-range instructions, and a grounding Transformer contextually represents UI objects using both their content and their screen position before connecting them to the extracted object descriptions. The decoupling of language and action data allows the two stages to be trained on different sources. *Results.* The model achieves 70% partial accuracy on the PixelHelp benchmark for instructions paired with starting screens, establishing a non-trivial baseline for grounded mobile UI control before the availability of LLMs. *Areas for further research.* The authors identify span identification, multitask learning, and reinforcement learning for out-of-sample grounding as directions, and explicitly note that the approach generalizes beyond Android to other UI platforms.

Group F: Vision-Language Foundation

[15] CogAgent: A Visual Language Model for GUI Agents (Hong et al., 2024). *Scope and dataset.* Presented at CVPR 2024, CogAgent is an 18-billion-parameter vision-language model specialized for GUI understanding and navigation, trained on a large-scale GUI dataset constructed by the authors. *Methodology.* The model uses both low-resolution and high-resolution image encoders to support 1120-by-1120 pixel input, enabling recognition of small icons and text typical of GUIs. A high-resolution cross-module branch handles the high-resolution path with controlled compute cost. The model is evaluated on text-rich VQA benchmarks and on GUI navigation benchmarks including Mind2Web and AITW. *Results.* CogAgent achieves state-of-the-art performance on five text-rich and four general VQA benchmarks and outperforms LLM-based methods that consume extracted HTML on PC and Android GUI navigation. The work supports the position that pure-vision GUI agents are viable when the underlying model is trained appropriately. *Areas for further research.* Sample efficiency, action-space coverage, and cross-platform generalization are identified.

Group G: Desktop and Cross-Application Layer

[16] ScreenAgent: A Vision Language Model-driven Computer Control Agent (Niu et al., 2024). *Scope and dataset.* ScreenAgent is a VLM-driven agent for cross-application desktop control. The

authors construct the ScreenAgent Dataset for training, capturing screenshot-action trajectories. *Methodology.* The agent operates in a planning-acting-reflecting cycle: it plans the next action from a screenshot observation, executes the action via emitted mouse and keyboard events, and reflects on the outcome before the next step. Actions are specified as coordinates and key sequences rather than as element indices. *Results.* ScreenAgent achieves performance comparable to GPT-4V with more precise UI positioning on its evaluation suite, demonstrating that mid-scale VLMs can support desktop computer-use agents. *Areas for further research.* Vision-only grounding is acknowledged as less reliable than approaches with API access; the integration of API and vision paths is identified as future work.

[17] WinClick: GUI Grounding with Multimodal Large Language Models (Hui et al., 2025).

Scope and dataset. WinClick targets Windows-desktop GUI grounding and introduces the WinSpot benchmark for Windows GUI grounding evaluation. *Methodology.* The system performs visual grounding from screenshots to detect actionable regions, with LLM-based alignment of the resulting grounding data. The authors observe that most existing GUI agents rely on structured data formats like DOM or HTML, which are inaccessible across general desktop environments, and design WinClick to operate without such structured data. *Results.* The grounding-pretrained MLLM substantially outperforms zero-shot MLLMs on Windows GUI tasks, establishing visual grounding as feasible for desktop without structured intermediates. *Areas for further research.* Cross-platform generalization (Linux, macOS), recovery from grounding errors, and integration with task-level planning are identified.

[18] UFO²: The Desktop AgentOS (Zhang et al., 2025). *Scope and dataset.* UFO² positions itself as a "Desktop AgentOS" for Windows, evaluated against task benchmarks across multiple Windows applications. *Methodology.* The system uses a hybrid action mechanism that fuses Windows UI Automation (the native accessibility framework) with vision-based parsing of the UI Automation tree. A HostAgent decomposes user tasks into subtasks dispatched to application-specialized AppAgents, each of which uses a unified GUI-API layer to execute through native APIs where available and through visual grounding as fallback. *Results.* UFO² substantially outperforms vision-only baselines on Windows desktop tasks, confirming the value of hybrid execution. The authors argue that existing CUAs are hindered by shallow OS integration, fragile screenshot-based interaction, and disruptive execution. *Areas for further research.* Linux and macOS coverage, sandboxing for safety, and full-OS conversational replacement (as opposed to atop-the-OS automation) are identified as next directions.

Group H: Evaluation and Synthesis

[19] OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments (Xie et al., 2024). *Scope and dataset.* Presented at NeurIPS 2024, OSWorld is a real-computer-environment benchmark covering Ubuntu, Windows, and macOS, with 369 real-world computer tasks where humans achieve 72.36% success while the best evaluated model reaches 12.24%, with weaknesses identified in GUI grounding and operational knowledge. *Methodology.* OSWorld provides a unified evaluation harness with task setup, execution, and verification across operating systems and applications, supporting agents driven by GPT, Claude, Gemini, Qwen, Mixtral, CogAgent, UI-TARS, Operator, and Agent-S. *Results.* The wide gap between human and best-model performance, sustained across model families, establishes that no current agent approaches human-level OS competence. *Areas for further research.* GUI grounding remains the primary failure mode; long-horizon planning, recovery from errors, and cross-OS generalization are identified.

[20] Large Language Model-Brained GUI Agents: A Survey (Zhang et al., 2024). *Scope and dataset.* This survey synthesizes the GUI-agent literature through 2024, organizing the field around components including perception, planning, action, and memory. *Methodology.* The work is a literature review rather than an empirical contribution, but it is foundational for any subsequent survey because it establishes shared vocabulary and identifies open problems. *Results.* The survey enumerates representative systems, classifies them along architectural and capability dimensions, and identifies open challenges including accurate element localization, knowledge retrieval, long-horizon planning, and safety-aware execution control. *Areas for further research.* Each enumerated open challenge motivates a research thread; full-OS conversational replacement, evaluated empirically, is among the gaps the survey leaves open and which SynapseOS addresses.

V. Analysis of Papers

A. Comparison Matrix

The matrix below positions all twenty surveyed works along the four-layer pipeline. Layers are abbreviated as Input (I), Reasoning (R), Action (A), and OS Integration (O).

Table 1: Comparative analysis of twenty surveyed works on conversational and LLM-mediated interfaces for personal computing, positioned along the four-layer pipeline (Input, Reasoning, Action, OS Integration), with coverage scope, action mechanism, evaluation, and key limitations.

No	Ref	System	Year	Layer Focus	Coverage Scope	Action Mechanism	Evaluation	Key Limitation
1	[1]	UNIX Consultant	1988	I, R, O	Shell consultation	Natural-language Q&A (no command exec)	Demo and case studies	Hand-engineered knowledge base
2	[2]	NL2Bash	2025	I, R, A	Shell (CLI)	Command translation	600-pair test set, fine-tuning benchmarks	Limited system context awareness
3	[3]	NaSh	2025	R, A, O	Shell (Linux)	Guarded command translation	Worked examples, design paper	No formal user evaluation
4	[4]	Multimodal GUI Architecture	2025	I, R, A	Single application	API binding via MCP + MVVM	Architectural prototype	Per-app retrofitting required
5	[5]	VoicePilot	2024	I, R, A	Assistive robot	Voice + Robot API	UIST user study with motor-impaired participants	Robot domain, not OS
6	[6]	DexAssist	2024	I, R, A	Web (e-commerce)	Dual-LLM voice + DOM	Web-task evaluation	Narrow to e-commerce
7	[7]	Mind2Web	2023	R, A	Web (137 sites)	Element selection from filtered HTML	NeurIPS dataset benchmark	HTML-only, no live evaluation in original
8	[8]	WebArena	2024	A, O	Web (4 self-hosted sites)	Functional task execution	ICLR benchmark, GPT-4 baseline	Self-hosted simulation, not live web
9	[9]	SeeAct	2024	R, A	Web (live sites)	Visual plan + grounding to HTML	ICML, 51.1% with oracle grounding	Grounding without oracle remains hard
10	[10]	Android in the Wild	2023	A	Mobile (Android)	Coordinate gestures	NeurIPS dataset, 715k episodes	UI-update brittleness
11	[11]	AppAgent	2023	R, A	Mobile (10 apps)	Tap/swipe + XML labels	50-task evaluation	Per-app knowledge acquisition needed
12	[12]	Mobile-Agent	2024	R, A	Mobile (vision-only)	OCR + icon detection + planning	Mobile-Eval benchmark	Vision-grounding errors
13	[13]	CogAgent	2024	R	Cross-platform GUI VLM	1120×1120 visual encoder	CVPR, SOTA on Mind2Web/AITW	Compute cost of high-res inputs
14	[14]	ScreenAgent	2024	R, A	Cross-app desktop	Vision + mouse/keyboard	Plan-Act-Reflect cycle,	Vision-only grounding fragility

No	Ref	System	Year	Layer Focus	Coverage Scope	Action Mechanism	Evaluation	Key Limitation
							ScreenAgent Dataset	
15	[15]	WinClick	2025	R, A	Cross-app desktop (Windows)	Visual grounding	WinSpot benchmark	Windows-specific
16	[16]	UFO ²	2025	R, A, O	Cross-app desktop (Windows)	Hybrid API + vision	System deployment with task benchmarks	Windows-only; not full-OS replacement
17	[17]	OSWorld	2024	(Eval)	Ubuntu, Windows, macOS	Evaluation harness	NeurIPS, 369 tasks, 12.24% best vs 72.36% human	Reveals field-wide grounding gap
18	[18]	LLM-Brained GUI Agents Survey	2024	(Survey)	Cross-platform	N/A	Literature synthesis	Synthesis only; no system contribution
19	[19]	PixelHelp / seq2act	2020	R, A	Mobile (Pixel)	Two-stage Transformer (extraction + grounding)	ACL benchmark, three datasets including PixelHelp	Pre-LLM; dataset-bound
20	[20]	ReAct	2023	R	Cross-domain (QA, web, games)	Prompting pattern (Thought-Action-Observation loop)	ICLR benchmarks across HotPotQA, Fever, ALFWorld, WebShop	Inherits LLM hallucination and long-horizon brittleness

B. Cross-Paper Patterns

Five patterns emerge from the matrix and from the per-paper analysis.

The first pattern is the bifurcation of the field by coverage scope. Shell-level work (NL2Bash, NaSh) and per-application work (the Multimodal GUI Architecture, VoicePilot) sit at the narrow end. Cross-application desktop work (ScreenAgent, WinClick, UFO²), web agents (Mind2Web, WebArena, SeeAct), and mobile agents (AITW, AppAgent, Mobile-Agent) sit at the broad end but each within a single platform class. No surveyed system implements a working full-OS conversational replacement that has been empirically validated against the conventional desktop workflow it would displace. The

Berkeley UNIX Consultant came closest in ambition four decades ago, but its pre-LLM technical foundation prevented it from scaling beyond demonstration.

The second pattern concerns the relationship between the Reasoning layer and the Action layer. The literature splits between three execution mechanisms — command translation, API binding, and GUI simulation — and the dominant trend is toward hybrid execution (UFO², the Multimodal GUI Architecture) that uses APIs where available and vision-based simulation as fallback. Vision-only systems (ScreenAgent, Mobile-Agent, CogAgent) achieve broad applicability at the cost of grounding fragility, and API-only systems (Mind2Web in its original form, the Multimodal GUI Architecture) achieve precision at the cost of per-environment customization. The Berkeley UNIX Consultant was unusual in operating purely at the consultation layer without executing actions, an approach that has not survived into the LLM era but which reflects an underappreciated design choice: the agent that explains is often safer than the agent that acts.

The third pattern concerns visual grounding, identified by both SeeAct and the GUI-Agents survey as the field's principal bottleneck. SeeAct demonstrates that GPT-4V with oracle grounding achieves 51.1% live-website task success but degrades substantially when grounding must itself be inferred. WinClick and follow-up work train dedicated grounding models. CogAgent extends the foundation-model approach with high-resolution visual encoders. The grounding problem is not solved by any surveyed system, and OSWorld's reported 12.24% best-model performance is attributable in significant part to grounding failure.

The fourth pattern concerns evaluation. Most surveyed systems are evaluated either through architectural demonstration (van Dam, NaSh) or through curated task benchmarks (Mind2Web, WebArena, OSWorld, Mobile-Eval). User studies are rare; only VoicePilot conducts a substantive human-centered evaluation of LLM-based interaction. No surveyed work compares its interface against the conventional graphical workflow for the same task on the same population. This is the empirical gap that would best be filled by a study of conversational OS interfaces.

The fifth pattern concerns the operating-system substrate. Windows is heavily represented (UFO², WinClick), Android is heavily represented (AITW, AppAgent, Mobile-Agent), and the web is heavily represented (Mind2Web, WebArena, SeeAct). Linux desktop coverage is conspicuously absent: NaSh addresses Linux but only at the shell level, OSWorld includes Ubuntu but only as one of three evaluation platforms, and no surveyed system implements a working full-OS conversational layer for

Linux. This is a substantive gap rather than an artifact of search bias, and it is the gap into which SynapseOS proposes to enter.

The sixth pattern concerns the field's intellectual lineage. The surveyed literature exhibits a clear before-and-after structure organized around the introduction of LLM-based reasoning. Pre-LLM work treated grounding as a supervised learning problem requiring purpose-built datasets and architectures: PixelHelp and its companion datasets [19], for example, trained dedicated phrase-extraction and grounding Transformers on hand-annotated mobile-UI action sequences. The introduction of in-context reasoning patterns, most notably ReAct [20], collapsed this multi-stage supervised pipeline into a prompting strategy executed by a single foundation model. Every system in Groups D, E, and G of this survey — Mind2Web, WebArena, SeeAct, AppAgent, Mobile-Agent, ScreenAgent, WinClick, and UFO² — inherits from ReAct the convention of interleaving reasoning traces with environment-affecting actions, and several of them explicitly cite the framework. The historical significance is twofold. First, it explains why progress in the field has been so rapid since 2023: the methodological foundations were portable across modalities and platforms in a way that the pre-LLM, dataset-bound approaches were not. Second, it underscores a corresponding fragility: nearly every surveyed agent inherits the failure modes of LLM reasoning — hallucinated tool calls, plan inconsistency, brittleness on long-horizon tasks — alongside its successes, which is the substantive concern that the OSWorld results in Section IV quantify.

C. Tradeoffs Identified

Three architectural tradeoffs recur across the surveyed literature. First, structured access (DOM, accessibility tree, native API) is faster and more reliable but requires per-application or per-platform integration; visual access generalizes but is less reliable. Second, narrow scope (one application, one platform) is empirically tractable but limits the practical value of the resulting interface; broad scope (full OS, cross-platform) is what users want but is empirically harder to validate. Third, executing actions on the user's behalf provides utility but introduces irreversibility and safety concerns; consulting without executing (the UC tradition) is safer but offers less value. SynapseOS, described next, takes positions on each of these tradeoffs.

VI. Proposed Conceptual Framework:

SynapseOS

SynapseOS is a Linux-based conversational interface layer intended to serve as the primary user-facing interface to a personal computing environment, replacing rather than augmenting the conventional graphical desktop. The framework follows the four-layer pipeline introduced in Section III and addresses the gaps identified in Section V.

A. Architecture Overview

SynapseOS sits between the user and the underlying Linux kernel and userspace. Its architecture comprises four cooperating components corresponding to the four pipeline layers.

At the **Input Layer**, SynapseOS accepts typed text, voice via on-device speech recognition, and structured shortcuts that allow power users to bypass natural language for high-frequency operations. Input modality is selectable per session and per task; the system is text-first by default but voice-capable by configuration, drawing on the design guidelines established by VoicePilot for users with motor accessibility needs.

At the **Reasoning Layer**, SynapseOS uses a multimodal LLM as its core interpreter. The LLM is provided with three context streams: the user's current intent, a structured representation of the system state (active applications, open files, recent commands) drawn from a context manager analogous to the agent-OS blueprints in the literature, and a tool catalog describing available actions exposed via MCP. The reasoning component decomposes the user's intent into a sequence of subgoals and grounds each subgoal into a concrete action, drawing on the SeeAct two-stage planning-then-grounding pattern but with structured-tool-first preference.

At the **Action Layer**, SynapseOS adopts the hybrid execution mechanism pioneered by UFO² but adapted to Linux. Three action paths are available, ordered by preference: (1) MCP-exposed application APIs, where applications have explicitly published their tools to the SynapseOS context manager; (2) Linux accessibility-tree access via AT-SPI, where applications have registered with the platform accessibility framework; (3) visual grounding plus simulated keyboard and mouse, as a last-

resort fallback. The first path is fastest and safest; the third is most general. SynapseOS attempts the paths in order and falls back gracefully.

At the **OS Integration Layer**, SynapseOS exposes itself as a Linux session manager, replacing the conventional desktop shell at startup. It provides a persistent conversation surface, command history, and a structured undo log that records all executed actions for rollback. Privileged operations are gated by a confirmation policy informed by NaSh's guardrail design: irreversible operations (file deletion, system configuration changes) require explicit user confirmation by default; reversible operations (window placement, application launch) execute directly.

B. Differentiators

SynapseOS occupies a position not held by any surveyed system. Compared to NaSh, SynapseOS extends from the shell to the full graphical session. Compared to UFO², SynapseOS targets Linux instead of Windows and aspires to replace the desktop rather than to automate atop it. Compared to the Multimodal GUI Architecture, SynapseOS operates at the OS level rather than within a single application. Compared to the Berkeley UNIX Consultant, SynapseOS executes actions rather than only consulting, while preserving the consultation mode as a fallback for ambiguous instructions and as a teaching mechanism for novice users.

C. Evaluation Plan

The evaluation gap identified in Section V motivates the central contribution of the SynapseOS thesis: a controlled comparison of SynapseOS against the conventional Linux desktop workflow on the same task suite, with the same participants, measuring task completion time, error rate, and self-reported satisfaction. The evaluation will draw on OSWorld's task suite where applicable for benchmark comparability, but will extend to a user study across at least two distinct user populations. The candidate populations are novice computer users (where conversational interfaces are hypothesized to outperform conventional GUIs) and power users (where the comparison is hypothesized to be less favorable but informative). A third population — users with motor-accessibility needs — would be the most distinctive contribution but is scoped out of the initial evaluation due to recruitment and ethics-review considerations.

D. Open Questions

Three open questions remain in the proposed framework. First, the choice of underlying multimodal LLM is not yet committed; whether to use a hosted frontier model with associated latency and privacy implications, or a locally-hosted open-weights model with associated capability tradeoffs, is a substantive design decision with implications for the evaluation. Second, the safety policy for irreversible actions has been described in principle but not specified at the level of individual command classes; which Linux operations require confirmation, which permit silent execution, and how the policy adapts to user expertise are open. Third, the relationship between SynapseOS and existing Linux desktop environments (GNOME, KDE) — whether SynapseOS replaces them entirely, runs alongside them as an alternative session, or is layered on top of one — is an integration question that bears on deployment but not on the conceptual framework.

References

- [1] Robert Wilensky, David N. Chin, Marc Luria, James Martin, James Mayfield, and Dekai Wu. 1988. The Berkeley Unix Consultant Project. *Computational Linguistics* 14, 4 (1988), 35–84. <https://aclanthology.org/J88-4003/>
- [2] Finnian Westenfelder, Erik Hemberg, Miguel Tulla, Stephen Moskal, Una-May O'Reilly, and Silviu Chiricescu. 2025. LLM-Supported Natural Language to Bash Translation. arXiv:2502.06858 [cs.CL]. <https://arxiv.org/abs/2502.06858>
- [3] Bimal Raj Gyawali, Saikrishna Achalla, Konstantinos Kallas, and Sam Kumar. 2025. NaSh: Guardrails for an LLM-Powered Natural Language Shell. arXiv:2506.13028 [cs.OS]. <https://arxiv.org/abs/2506.13028>
- [4] Hans G. W. van Dam. 2025. A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants. arXiv:2510.06223 [cs.HC]. <https://arxiv.org/abs/2510.06223>
- [5] Akhil Padmanabha, Jessie Yuan, Janavi Gupta, Zulekha Karachiwalla, Carmel Majidi, Henny Admoni, and Zackory Erickson. 2024. VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (UIST '24). ACM, New York, NY, USA. <https://doi.org/10.1145/3654777.3676401>
- [6] Shridhar Mehendale and Ankit Walishetti. 2024. DexAssist: A Voice-Enabled Dual-LLM Framework for Accessible Web Navigation. arXiv:2411.12214 [cs.HC]. <https://arxiv.org/abs/2411.12214>
- [7] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems* 36 (NeurIPS 2023) Datasets and Benchmarks Track. <https://arxiv.org/abs/2306.06070>
- [8] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of the 12th*

International Conference on Learning Representations (ICLR 2024).
<https://arxiv.org/abs/2307.13854>

- [9] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. <https://arxiv.org/abs/2401.01614>
- [10] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2023. Android in the Wild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023) Datasets and Benchmarks Track*. <https://arxiv.org/abs/2307.10088>
- [11] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2023. AppAgent: Multimodal Agents as Smartphone Users. arXiv:2312.13771 [cs.CV]. <https://arxiv.org/abs/2312.13771>
- [12] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. arXiv:2401.16158 [cs.CV]. <https://arxiv.org/abs/2401.16158>
- [13] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2024)*. <https://arxiv.org/abs/2312.08914>
- [14] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. 2024. ScreenAgent: A Vision Language Model-driven Computer Control Agent. arXiv:2402.07945 [cs.CV]. <https://arxiv.org/abs/2402.07945>
- [15] Taiwei Hui, Yiwen Zhang, Bin Jiang, Jianhua Sun, Fan Cheng, Xinya Lin, and Yi Yang. 2025. WinClick: GUI Grounding with Multimodal Large Language Models. arXiv:2503.04730 [cs.HC]. <https://arxiv.org/abs/2503.04730>
- [16] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. UFO²: The Desktop AgentOS. arXiv:2504.14603 [cs.AI]. <https://arxiv.org/abs/2504.14603>

- [17] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37* (NeurIPS 2024). <https://arxiv.org/abs/2404.07972>
- [18] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2024. Large Language Model-Brained GUI Agents: A Survey. arXiv:2411.18279 [cs.AI]. <https://arxiv.org/abs/2411.18279>
- [19] Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge. 2020. Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (ACL 2020). Association for Computational Linguistics, 8198–8210. <https://doi.org/10.18653/v1/2020.acl-main.729>
- [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations* (ICLR 2023). <https://arxiv.org/abs/2210.03629>